

Performance and Energy Consumption in Mobile Applications: A Flutter Perspective

Module 1: The Foundations of Mobile Performance

What is Performance?

In the domain of mobile applications, "performance" is a multifaceted concept that extends beyond simple speed. It is a comprehensive measure of user-perceived quality, which can be delineated into two primary, and sometimes conflicting, pillars:

1. **Speed (Responsiveness):** This pillar addresses the app's ability to react to user input and deliver content. It encompasses metrics such as application startup latency, the smoothness of scrolling and animations, and the responsiveness of UI elements. This is a measure of both latency and throughput.
2. **Efficiency (Resource Use):** This pillar concerns the app's consumption of finite system resources. Key considerations include the application's memory (RAM) footprint ¹, its disk storage size, and, critically, its impact on battery life through energy consumption.

A successful, high-performance application must achieve a balance between these two pillars. An application that is exceptionally fast but drains the device's battery rapidly is considered a low-performance app by the end-user, who is 2.5 times more likely to uninstall such an application.² Similarly, an efficient app that feels sluggish or unresponsive fails to meet user expectations. This lecture will explore the measurement, diagnosis, and optimization of both pillars, with a specific focus on their application within the Flutter framework.

Core Performance Metric 1: Startup Time

An application's startup time is its first impression on the user. It is defined as the duration from the moment the user taps the application icon to the moment the app's data is visible and ready for interaction.³ A slow startup creates a perception of a "heavy" or poorly

optimized application.

Startup time is not a monolithic metric. It is categorized into three distinct types, each defined by the state of the application's process in the system's memory.³

- **Cold Start:** This is the slowest launch scenario, occurring when the application's process is not already present in the system's memory.³ This happens on the app's first launch after a device reboot or after the system (or user) has terminated the app's process, often to reclaim resources.⁵ During a cold start, the system must perform the maximum amount of work: loading the application's executable from storage, creating a new process, initializing the runtime (e.g., Android Runtime or Dart VM), and creating and rendering the initial UI.⁴
- **Warm Start:** This is a partial startup. In this scenario, the application's process is already running in the background, but the specific UI (e.g., the Android Activity) has been destroyed and must be recreated.³ This is significantly faster than a cold start because the system reuses the existing process, and some data or state may still be resident in memory.⁷
- **Hot Start:** This is the fastest and most common scenario, such as when a user switches between recent apps. The application's process *and* UI are already in memory, running in the background.³ The system simply brings the app to the foreground, resulting in minimal overhead.⁵

The distinction between platform-defined *penalty thresholds* and aggressive *engineering goals* is critical. While Google's Android Vitals program flags an app for "excessive" cold startup only if it exceeds 5 seconds⁶, the ideal engineering target for a high-quality application is less than 500ms.³ This 4.5-second gap represents a "zone of mediocrity": an app with a 4-second cold start is not penalized by the platform but is delivering a user experience that is eight times slower than the ideal target.

Start Type	System State	Process Status	Ideal Goal (Engineering)	Android Vitals Threshold (Penalty)
Cold	App not in memory.	Process must be created, runtime initialized, UI rendered.	$\$ < 500 \text{ms} \$$ ³	$\$ > 5 \text{s} \$$ ⁶
Warm	App process in memory.	Process is reused, UI must be recreated.	(Not specified)	$\$ > 2 \text{s} \$$ ⁶
Hot	App process and UI in memory.	System brings app to foreground.	(Not specified)	$\$ > 1.5 \text{s} \$$ ⁵

Core Performance Metric 2: Rendering and Frame Rate

Once an application is running, the performance focus shifts from latency to *smoothness*. This is dictated by the rendering pipeline's ability to produce and display frames consistently.⁹ The goal is to achieve a "buttery smooth" experience, which occurs when the application's **Frames Per Second (FPS)** match the device's display **refresh rate**.⁹

For the majority of mobile devices, the display refresh rate has historically been **60Hz**, or 60 cycles per second.³ This imposes a non-negotiable temporal deadline on the application:

$$1 \text{ second} / 60 \text{ frames} = 16.67 \text{ milliseconds per frame}$$

This 16.67ms window (commonly rounded to 16ms) is the application's **frame budget**.¹⁰ All work required to produce a single frame—executing application logic, performing UI layout, and painting (rendering) to the screen—must be completed within this budget.

This challenge is escalating. Modern devices increasingly feature high-refresh-rate screens, such as **90Hz** or **120Hz**.³ These devices raise the performance bar significantly, as the frame budget is proportionally reduced:

- **90Hz:** $1000 \text{ ms} / 90 \text{ frames} = 11.1 \text{ ms}$ per frame
- **120Hz:** $1000 \text{ ms} / 120 \text{ frames} = 8.3 \text{ ms}$ per frame

This shift represents a fundamental challenge to mobile performance. An application that was perfectly optimized for 60Hz—for example, one that consistently renders frames in 12ms—will now appear *janky* on a 120Hz device because it is missing the 8.3ms budget. Complacency with a 16ms target is no longer sufficient.

Core Performance Metric 3: Understanding Jank

The term **jank** describes the "visual hiccup, stutter, or pause" that a user perceives when interacting with an app.³ It is the direct, user-visible consequence of an application *missing* its frame budget. If a frame takes 30ms to render instead of the allotted 16ms, the system is forced to *drop* a frame. The previous frame is held on-screen for an additional 16ms, causing a noticeable "freeze" or stutter.¹⁰

A common misconception is that a low *average* FPS causes jank. This is incorrect. A game running at a *consistent* 30 FPS can feel smooth, while an application with an *average* of 55 FPS can feel terrible. The reason for this paradox is *variance*.

Performance analysis must therefore focus on **Frame Time** (or "frametime") rather than average FPS.¹³ Frame time measures the duration of *each individual frame*. Jank is a problem of *frame time variance*, not average frame rate.

An analysis of two applications illustrates this point perfectly¹³:

- **App A**¹³ had an average FPS of 40. However, it experienced a single frame that took 117ms to render. This massive spike is a user-visible jank that shatters the illusion of smoothness.
- **App B**¹³ had a lower average FPS of 30. However, its frame times were highly *consistent*, hovering around 33ms. This application feels *smoother* to the user

than App A.

The key takeaway is that *average* FPS is a misleading, and often useless, vanity metric. The critical metric for diagnosing jank is the *variance* in frame time. As professional-grade tools confirm, "RenderTime metrics... are more important than the count of janky frames".³ The goal is a *flat, consistent* frame time graph, not a high average FPS.

Core Performance Metric 4: Memory Footprint

Random-access memory (RAM) is a highly valuable, constrained, and shared resource in a mobile operating system.¹ An application's memory footprint has a direct and profound impact on overall system performance, connecting back to our other core metrics.

If an application consumes an excessive amount of memory, two negative outcomes occur¹:

1. **Poor System Experience:** To reclaim memory for the foreground app, the operating system will terminate processes running in the background (e.g., a user's music player or a navigation app).
2. **Forced Cold Starts:** If the application itself is in the background and consumes too much memory, the system will terminate *its* process. This means the next time the user opens the app, they are forced to experience a slow *cold start* instead of a fast hot or warm start.

Memory management strategies differ significantly between the two major native platforms, and this distinction is crucial for understanding cross-platform frameworks:

- **iOS (Swift/Objective-C):** Uses **ARC (Automatic Reference Counting)**. ARC is a compile-time mechanism. Memory for an object is released *deterministically* and *immediately* at the moment its reference count drops to zero.
- **Android (Kotlin/Java):** Uses **Garbage Collection (GC)**. GC is a *non-deterministic*, runtime process.¹⁴ The garbage collector must periodically run, *pause* the application (or its threads), scan the memory heap for objects that are no longer referenced, and then free them.¹

This architectural difference has two major performance consequences. First, GC-based systems tend to have a higher, less efficient memory footprint because objects are not released immediately; they "linger" until the next GC cycle.¹⁴ Second, the GC process itself can be a direct cause of *jank*. These GC "pauses" can "stop-the-world," stealing time from the application's 16ms frame budget and causing a stutter.¹ Thus, an application with a high rate of memory allocation (a *memory* problem) can manifest as a *jank* problem (a *speed* problem).

Module 2: The Silent Drain: Understanding Energy Consumption

Why Energy Consumption Matters

The second pillar of performance is *efficiency*, and its most user-facing metric is energy consumption, or battery life. On desktop platforms, software developers traditionally ignored energy consumption. In the mobile environment, it is a *critical* component of the user experience.¹⁵

User retention is directly tied to an app's energy profile. Data shows that applications perceived as "battery hogs" are uninstalled 2.5 times more frequently than their efficient counterparts.²

Energy consumption is not an abstract hardware problem; it is a direct, measurable consequence of software design. Every time an application performs a heavy computation (CPU/GPU load), keeps the screen bright, requests data over the network, or polls the GPS, it is actively making a decision to drain the battery. The role of the developer is to understand the *energy cost* of these operations and to use "energy profiling tools" with the same diligence as performance profiling tools.¹⁵

Primary Energy Driver 1: The Screen

The single largest consumer of energy in a mobile device is the display.¹⁶ The primary variable controlling this drain is screen brightness, but *how* energy is consumed is entirely dependent on the underlying display technology.

- **LCD (Liquid Crystal Display):** LCDs use a single, uniform backlight to illuminate all pixels. The pixels themselves act as shutters, blocking or allowing light to pass. Consequently, power draw is related *only* to the *brightness level* of the backlight, not the colors being displayed. At 50% brightness, a pure black screen and a pure white screen consume nearly identical amounts of power.
- **OLED / AMOLED (Organic Light-Emitting Diode):** In this technology, every pixel is its own individual light source. Power consumption is therefore a function of *both* brightness *and* the color of each pixel. A white pixel is fully "on" and consumes power. A black pixel is "off" and consumes *zero* power.

This technological difference is the key to understanding the "Dark Mode" debate. A "dark mode" theme will *not* provide significant battery savings on an **LCD** screen, as the backlight's power draw remains constant. However, on an **OLED** screen, a true-black dark mode results in *massive* energy savings, as a large percentage of the display's pixels are turned completely off.¹⁸ This effect is magnified at higher brightness levels.¹⁸

For all screen types, "auto-brightness" features are an energy win. They prevent the worst-case scenario: a user leaving the screen at 100% brightness in a dark environment, which provides no user benefit and drains the battery rapidly.¹⁹

Primary Energy Driver 2: CPU and GPU Load

The second major energy drain is computation. Every instruction executed by the Central Processing Unit (CPU) and every pixel drawn by the Graphics Processing Unit (GPU) consumes power.¹⁶ High CPU/GPU load is a direct result of software decisions, including:

- **Inefficient Code:** A poorly-written algorithm (e.g., $O(n^2)$ where $O(n)$ is possible) forces the CPU to run for a longer duration, consuming more total energy.
- **Heavy Animations:** Constant, looping animations force the GPU to redraw the screen 60 or 120 times per second, even when the application is ostensibly "idle".²¹
- **Graphics-Intensive Tasks:** Obvious loads include gaming²², 3D rendering, and video processing or streaming.²³

This computational load generates *heat* as a byproduct. The system must dissipate this heat, either by using fans (on laptops)²⁴ or, more commonly on phones, by **thermal throttling**.²⁵ Throttling is a self-preservation mechanism where the OS *slows down* the CPU to reduce heat and prevent damage.²⁵

This creates a vicious cycle that connects energy and performance.

1. An energy-inefficient algorithm (a "slow" problem) runs the CPU at 100%.
2. The CPU overheats.
3. The OS throttles the CPU to cool it down.
4. The *exact same code* that previously took 12ms to run now takes 20ms .
5. The application has become *janky* (a "fast" problem), not because the code changed, but because its *energy inefficiency* created a *thermal* problem, which in turn created a *jank* problem.

Primary Energy Driver 3: The Network Radio

One of the most significant and non-obvious energy drains is the network radio—the hardware component responsible for cellular (3G/4G/5G) and Wi-Fi communication.²⁶ The problem is not simply the *amount* of data transferred, but the *frequency* of network access. To conserve power, mobile radios implement a **Radio Resource Control (RRC) state machine**.²⁷ A simplified model of this machine includes:

- **IDLE (Low Power):** The radio is asleep, consuming almost no power.
- **DCH (Dedicated Channel / High Power):** The radio is fully active, with a reserved channel for high-throughput, low-delay transmission. This state consumes a very high amount of power.²⁷

The performance trap lies in the transition *out* of the high-power state. After an application finishes a data transfer (e.g., sending a 1KB analytics packet), the radio does *not* immediately return to the IDLE state. It remains in the high-power DCH state for a fixed **inactivity timer** (e.g., 5-10 seconds) in case more data is coming.²⁷

This creates an **"energy tail"**: a long period of high-power consumption *after* the data transfer is complete.²⁷ This has catastrophic implications. A "chatty" application that sends a tiny analytics packet every 10 seconds is *continuously resetting* this inactivity timer. The radio is *never* allowed to return to the IDLE state. The developer believes they are paying the energy cost of a 1KB transfer, but they are *actually* paying the cost of holding the device in a high-power state 24/7, which will destroy the user's battery.

Primary Energy Driver 4: Location and Sensors

The final major energy driver is the sensor array, dominated by the **Global Positioning System (GPS)**.²⁸ The power drain from GPS is often misunderstood; it is not a simple, passive receiver.

The high cost of GPS is twofold ²⁹:

1. **Continuous Hardware + System State:** The GPS chip must be continuously powered to scan for and maintain locks on weak satellite signals.²⁸ More importantly, this continuous operation forces the *entire system* (including the CPU) into a high-power state and prevents it from entering low-power sleep.
2. **Continuous Computation:** Receiving the signals is only the first step. The CPU must then perform "extremely expensive" calculations (trilateration) to determine the device's precise location from those signals.²⁹

The key variable controlled by the developer is *frequency*. Requesting a single location update is a low-cost operation. Requesting "real-time tracking" (e.g., one update per second) is a "continuous operation" that combines all of these costs and is one of the fastest ways to drain a battery.³⁰ The "shorter the interval" for updates, the more the battery will be drained.³⁰

Module 3: The Flutter Architectural Model

How Flutter Changes the Equation

With a foundation in general mobile performance, it is time to introduce Flutter. Flutter's architectural model is fundamentally different from other cross-platform solutions (like React Native), which typically act as a *bridge*, translating application code (e.g., JavaScript) into calls that create *native* platform widgets (e.g., Android's TextView or iOS's UIButton). Flutter **does not** do this. This is the single most important concept in understanding Flutter's performance.

Flutter bypasses the platform's native UI widget library entirely. It ships its *own* rendering engine—historically **Skia**, and now **Impeller**—and uses it to paint *every single pixel* on the

screen.³² When a Flutter developer uses a Text widget, they are *not* creating a native TextView. They are issuing commands to the Flutter Engine, which tells the Skia/Impeller canvas to draw those specific letterforms to the screen.³²

This "Flutter owns the pixels" model is the source of both its greatest strengths and its most common performance traps:

- **Advantages:**
 - **Performance:** Code is AOT-compiled to native machine code (ARM/x64), not interpreted through a bridge.³⁴
 - **Consistency:** The UI is pixel-perfect identical on all platforms and OS versions because it is being drawn by the *same* engine, not by different sets of native widgets.
- **Disadvantages (Performance Traps):**
 - Because Flutter is not using the native widgets (which have been platform-optimized for a decade), it is now *100% responsible* for the performance of its *own* widget set.
 - An inefficient widget in Flutter (like Opacity, as will be discussed) is a performance bottleneck *everywhere*, and the developer cannot rely on native platform optimizations.

The Flutter Rendering Pipeline (Part 1): The Three Trees

To understand how Flutter renders, one must understand its "Three Trees" model, which separates configuration, state, and rendering.³⁵

1. **Widget Tree:** This is the tree a developer creates in their code. It is an **immutable configuration** or "blueprint" for the UI.³⁶ When a build() method is called, it returns a new Widget Tree (or subtree).³⁷ These are just lightweight Dart objects, making them extremely "cheap" to create and destroy.
2. **Element Tree:** This is the *living, mutable* counterpart to the Widget Tree. For every widget in the tree, the framework creates a corresponding Element *once*. This Element holds the *state* of the widget, its reference in the hierarchy, and its link to the underlying render object.³⁵ This tree is the *mediator*. When a developer calls setState(), the framework rebuilds the (cheap) Widget Tree and then walks the (persistent) Element Tree, comparing the new widget configuration to the old one.
3. **Render Tree (RenderObject Tree):** This tree does the *actual work* of layout and painting.³⁶ Only Elements that draw to the screen (specifically, RenderObjectElements) create a RenderObject.³⁵ The RenderObject is what computes layout constraints, determines its size, and implements the paint() method that draws to the canvas.³⁸

This separation is the core of Flutter's UI performance. The build() method can be called 60 times a second because it is only creating lightweight, immutable Widget "blueprints." The framework's entire optimization strategy is to use the Element tree to compare these blueprints and *avoid* touching the *expensive* RenderObject tree unless absolutely necessary.

The Flutter Rendering Pipeline (Part 2): Skia

When a `RenderObject` needs to paint, it issues commands to a graphics engine. For most of its history, Flutter has used **Skia**, the mature, open-source 2D graphics library that also powers Google Chrome and Android's native UI toolkit.³²

The "Flutter Engine" (a C++ application) acts as the intermediary. It takes the high-level drawing commands from the Render Tree (e.g., "draw this Text here," "paint this Container's shadow") and translates them into low-level Skia canvas commands.³² Skia then executes these commands, using the device's **GPU** for hardware-accelerated rendering, which enables smooth animations and high-performance graphics.³²

Despite its power, Skia had one infamous, critical flaw in the context of Flutter: **runtime shader compilation**.³⁹

A "shader" is a small program that runs directly on the GPU to create complex visual effects (e.g., blurs, gradients, opacity masks). To render an effect for the *first time*, Skia had to *compile* this shader program *on-the-fly*, at runtime. This compilation step is very slow, often taking many milliseconds—far longer than the 16ms or 8.3ms frame budget. This was the source of the "first-run jank" that plagued early Flutter applications. The *first time* a user opened a dialog with a blur effect, or the *first time* an animation with a gradient appeared, the app would stutter (jank) as the main thread was stalled, waiting for Skia to compile the necessary shader.³⁹

The Flutter Rendering Pipeline (Part 3): Impeller

To permanently solve the shader compilation jank problem, Google built a new rendering engine from the ground up, designed exclusively for Flutter: **Impeller**.⁴⁰ Impeller is now the default renderer on iOS and is being rolled out on Android.⁴⁰

Impeller's core objective is **"Predictable performance"**.⁴⁰ It achieves this by *completely eliminating* runtime shader compilation.

Impeller's architecture shifts this work from *runtime* to *compile time*.

1. It defines a smaller, simpler set of shaders.
2. These shaders are all **pre-compiled** at *engine-build time* (i.e., when the developer builds their app).⁴⁰
3. All "pipeline state objects" (PSOs) are built upfront.

At runtime, when an effect is needed, the engine no longer says, "please compile this shader code." It says, "run pre-compiled shader #5." This is a fundamental architectural shift. By trading the runtime flexibility of Skia's on-the-fly compiler for the "predictable performance" of a pre-compiled library, Impeller *guarantees* that shader compilation will never be a source of jank.⁴⁰ As confirmed by the Flutter team, "you should see the architectural benefits. There

is no jank due to shader compilation".⁴¹ If an application is still exhibiting "first-run jank," the primary recommendation is to ensure it is running on Impeller.⁴²

Flutter's Compilation Model: JIT vs. AOT

The final piece of Flutter's architecture is its dual compilation model. Dart, the language Flutter uses, is unique in its ability to be compiled in two distinct ways, which Flutter leverages for development and production.³⁴

1. JIT (Just-In-Time) Compilation:

- **Use Case:** Development (flutter run).
- **How it works:** The app runs inside a Dart Virtual Machine (VM) on the device.³⁴ This VM enables "**Stateful Hot Reload**," which allows developers to inject new code into the running app for an instant development cycle.³⁴
- **Performance:** Slower startup⁴³, higher memory footprint⁴⁴, and not indicative of release performance.

2. AOT (Ahead-Of-Time) Compilation:

- **Use Case:** Production (flutter build).
- **How it works:** The Dart code is compiled *directly* into native machine code (ARM or x64).³⁴ There is no VM or interpreter in the release app; it is a native executable.
- **Performance:** Fast startup⁴³, lower memory footprint⁴³, and predictable runtime performance.

Aspect	JIT (Flutter Debug)	AOT (Flutter Release)
Startup Time	Slower. VM must initialize, code is compiled/interpreted at runtime. ⁴³	Faster. Pre-compiled native code executes immediately. ⁴³
Runtime Performance	Potentially <i>higher</i> over time (with a mature JIT), as it can perform runtime optimizations (PGO) based on live data. ⁴⁵	Consistent & Predictable. Lacks runtime data for PGO, but optimized at build-time. ⁴³
Memory Usage	Higher. Requires the VM, JIT compiler, and intermediate code. ⁴³	Lower. Only the compiled machine code is needed. ⁴³
Primary Use	Development (Stateful Hot Reload) ³⁴	Release (User-facing app) ³⁵

A common misconception is that AOT-compiled code is *always* faster than JIT-compiled code.⁴⁵ A mature JIT compiler can, in fact, produce *faster* code over a long-running session because it can perform "Profile-Guided Optimizations" (PGO) based on *how the code is actually being used*—something an AOT compiler can only guess about.⁴⁶

Flutter's use of AOT for release is a *deliberate trade-off*. It *forges* the potential long-running

optimizations of a JIT in exchange for the *guaranteed* benefits of AOT, which are *far more important* in the mobile environment: **fast, predictable startup time** and a **low memory footprint**.⁴³ In an environment where a 5-second cold start is a "penalty"⁶, this is the correct engineering choice. Flutter gets the best of both worlds: JIT for unparalleled developer productivity, and AOT for high-performance, shippable applications.

Module 4: Diagnosing Performance Bottlenecks in Flutter

Why is my Flutter app janky? - Common Culprits

Given the $\$16\text{ms}$ (or $\$8.3\text{ms}$) frame budget⁴⁸, missing this target causes jank. In Flutter, these jank-inducing bottlenecks almost always fall into one of the following categories³⁹:

1. **Inefficient `setState()`:** Rebuilding *far more* of the UI than necessary, wasting CPU cycles in the `build()` method.³⁹
2. **Large/Deep Widget Trees:** Creating an overly complex widget hierarchy, which slows down the *Layout* and *Paint* phases.⁵⁰
3. **Expensive Widgets:** Using specific widgets like `Opacity` or `ClipRRect` without understanding their high *rendering* cost.⁵³
4. **List/Grid Inefficiencies:** Building long lists by passing children directly, rather than using the lazy-loading `ListView.builder`.⁴²
5. **Main Thread Blocking:** Performing heavy, synchronous work (like JSON parsing or database queries) on the main UI thread, *freezing* the app entirely.³⁹
6. **Platform Channel Overhead:** Communicating with native code at a high frequency, creating a bottleneck in data serialization.⁵⁶

Bottleneck 1: Inefficient `setState()` Calls

This is the most common performance trap for new Flutter developers. The `setState()` method, when called on a `State` object, signals to the framework that it needs to rebuild. The framework responds by calling the `build()` method on *that widget and ALL of its descendants*.⁵⁰

The performance problem arises when `setState()` is called *high* in the widget tree—for example, on the root widget of a screen.⁵⁰ If an application's entire page logic is in one `StatefulWidget`, and a 'like' button at the bottom calls `setState()` on that page, the framework

is forced to *rebuild the entire screen*: the AppBar, the body, the ListView, and every single item in it... just to update a 'like' count.⁵⁸

This practice of "rebuilding entire screens for small updates" ⁵⁸ is wildly inefficient. It wastes CPU cycles, burns the frame budget, and is a primary cause of jank.⁵¹ A build() method should be cheap, "capable of being performed 60 times per second".⁵⁹

The solution is encapsulated in this rule: **"Localize the setState() call to the part of the subtree whose UI actually needs to change"**.⁵⁰ If state must be shared more broadly, a simple setState() call is the wrong tool. This is a problem of *architecture*, which must be solved with a state management solution (e.g., Provider, BLoC) designed for granular, selective rebuilding.⁵⁸

Bottleneck 2: Large and Deep Widget Trees

The cost of a frame is not just in the build() method (Dart execution). The *Layout* and *Paint* phases, handled by the RenderObject tree, also have a cost.⁴⁹ This cost becomes problematic when widget trees are either "too deep" or "too large."

- **Deep Widget Trees:** Flutter's "everything is a widget" model encourages composition, which often leads to deep nesting: `Row(children: [Container(child: Column(children: [Padding(...)]))])`.⁵² While this is idiomatic, every level of nesting adds overhead to the *layout pass*. The RenderObjects must communicate constraints *down* the tree and sizes *up* the tree. In an extremely deep tree, this communication (while $O(N)$) can accumulate enough constant-time overhead to miss the frame budget.⁵¹ The solution is to refactor, breaking complex UIs into smaller, reusable widgets.⁵⁰
- **Large Widget Trees:** This is a far more severe problem. This occurs when a developer builds a large list of children directly, for example: `Column(children:)`. This is a performance disaster.⁴² The layout engine must instantiate, measure, and position *all* 1000 children, even the 990 that are off-screen. This will cause a multi-second freeze and consume a massive amount of memory.

The solution to "deep" trees is refactoring.⁵⁰ The solution to "large" trees is non-negotiable: **never** build a large list of children directly. **Always** use a constructor like `ListView.builder`.⁴² For *truly* massive widget counts (e.g., a hexagonal grid), even `ListView.builder` may be too slow, and a developer may need to drop down to a `CustomPaint` widget, bypassing the widget/element/render trees entirely for those items.⁶¹

Bottleneck 3: The Cost of Opacity and ClipRect

Some widgets are "deceptively expensive"—they appear simple but incur a significant performance cost. The most notable examples are Opacity and clipping widgets (`ClipRect`, `ClipPath`, etc.).⁵³

These widgets are not slow because of Dart code. They are slow because of *what they force the GPU to do*.

When Flutter encounters an Opacity widget, it cannot simply tell the GPU "draw these children at 50% transparency." If it did, overlapping children would show through each other. Instead, it must perform a **saveLayer()** operation.⁵⁰ This is an extremely "GPU-intensive" ⁵³ multi-step process:

1. The engine allocates a new, off-screen buffer in GPU memory.
2. It temporarily *switches* its rendering target to this new buffer.
3. It renders the *entire* child subtree into this off-screen buffer at 100% opacity.
4. It *switches* the rendering target back to the main screen.
5. It applies the opacity (a transparency shader) to the *entire off-screen buffer* as a single texture and composites it back onto the screen.

This process of allocating new GPU memory and switching rendering contexts is a major-league bottleneck. Clipping widgets trigger the same expensive "off-screen buffer" process.⁵³

Using one Opacity or ClipRRect widget on a static page is fine. Using one *inside an animation* (like a FadeTransition) or, far worse, *inside every item of a ListView*, is a primary and common cause of severe jank.

Bottleneck 4: ListView vs. ListView.builder

This is a "cardinal sin" of Flutter performance, but it is a direct application of the "Large Widget Tree" bottleneck.

- **The Problem:** ListView(children: [... 1000 items...]). The default ListView constructor requires a List<Widget>. It *builds all items at once*.⁵⁵
- **The Consequence:**
 1. **CPU Jank:** The build() method for the page will take hundreds of milliseconds (or seconds) as it constructs all 1000 widgets, *freezing* the app.
 2. **Memory Bloat:** All 1000 widgets are held in memory, even though only 10 are visible.⁵⁵ This massive memory footprint makes the app a prime target for being killed by the OS in the background, which in turn leads to more *cold starts*.
- **The Solution:** ListView.builder(). This constructor is the *only* correct way to build long or dynamic lists.⁵⁵ It implements **lazy loading**:
 - The itemBuilder function is called *only* for items that are about to scroll onto the screen.⁵⁵
 - Widgets that scroll off-screen are recycled.
 - This keeps both CPU usage (for building) and memory usage *low and constant*, regardless of whether the list has 100 items or 10,000.

Bottleneck 5: Platform Channel Overhead

When Flutter (Dart code) needs to access a native platform API (Kotlin/Swift code), such as the battery level or GPS, it uses **Platform Channels** (specifically `MethodChannel`).⁶³

This is *not* a "free" function call. It is an **asynchronous message-passing** system.⁵⁷ The performance cost is not in the call itself, but in the *data serialization*.

When Dart calls a method channel:

1. The Dart code *encodes* its arguments into a binary, JSON-like message.⁵⁷
2. This message is sent *asynchronously* from the Dart VM (UI thread) to the native platform's main thread.⁶³
3. The native (Kotlin/Swift) code *receives* this message and *decodes* (deserializes) it.
4. The native code executes its logic.
5. The native code *encodes* a response, which is sent back to Dart.
6. The Dart code *decodes* the response.

This "overhead" ⁵⁶ of encoding and decoding on both sides is negligible for infrequent calls (e.g., "get battery level once"). However, it is a *major bottleneck* for high-frequency data.⁵⁶ Attempting to stream 60fps camera data over a `MethodChannel` would fail, as the serialization overhead on *every frame* would guarantee missing the 16ms budget.

For true, high-performance synchronous interop, developers must use Dart's **FFI (Foreign Function Interface)**, which can call C-style native functions *directly* without any message passing or serialization overhead.⁵⁷

Module 5: Practical Performance Optimization in Flutter

Solution 1: const and RepaintBoundary

Identifying bottlenecks is the first step; fixing them is the second. These are two of the most powerful tools for doing so.

1. Use const... everywhere.

This is the easiest, cheapest, and most important performance optimization in Flutter.⁶⁶ It is the primary mechanism by which a developer leverages the three-tree architecture.

When the framework walks the Element tree to compare the new Widget tree to the old one, it checks for widget equivalency. If a developer uses a const constructor (e.g., `const Text('Hello')`), they are creating a **compile-time constant**.⁶⁹ This object is *canonicalized*; any `const Text('Hello')` in the code refers to the *exact same instance* in memory.⁶⁹

When the `build()` method returns this const widget, the framework sees that the *new* widget is *the exact same instance* as the *old* widget.⁵⁰ It therefore *immediately stops traversing* that

branch of the widget tree. It "**short-circuits**" the rebuild work, telling the framework, "this entire subtree is identical to the last frame, do not check its children, do not mark it for repainting".⁵⁰ This saves invaluable CPU time and reduces work for the garbage collector.⁷⁰

2. Use RepaintBoundary... strategically.

A RepaintBoundary is a widget that creates a separate rendering layer for its child subtree.⁷¹ It caches the painted output of its children.

- **Use Case:** Imagine a complex, static AppBar with an animated, scrolling ListView below it. Without a boundary, the AppBar might be repainted (even if its content doesn't change) because it's part of the same layer as the moving list. By wrapping the static AppBar in a RepaintBoundary, the framework paints it *once*, caches the result, and then for subsequent frames, it just re-uses the cached bitmap, only repainting the (separate) list layer.⁷³
- **The Trap:** This optimization must *not* be overused.⁷¹ A RepaintBoundary is a *cache*, and a cache *consumes memory*—in this case, GPU memory, which is a scarce resource.⁷⁴ If a RepaintBoundary is used to wrap a *constantly changing animation*, the cache is invalidated *every single frame*. The developer has gained *zero* caching benefit, all while *adding* the overhead of a new layer and *increasing* memory consumption, resulting in *worse* performance.⁷¹

The rule: Use RepaintBoundary to separate *static* content from *dynamic* content that is nearby in the widget tree.

Solution 2: Efficient ListView.builder

Using ListView.builder is the first step.⁶² The second is to optimize it further.

When a ListView scrolls, the layout engine does not know the size (height) of the widgets that are about to scroll onto the screen. It must *wait* for the itemBuilder to build the widget, *then* run that widget's layout() method, *then* get its computed height, and *only then* can it correctly calculate the scroll offset and position the next item. This "build-then-measure" process can cause jank during a fast scroll.

The solution is to *tell the ListView the size of its children ahead of time*. This can be done in two ways⁶²:

1. **itemExtent:** (e.g., itemExtent: 75.0). This property is used when *all* list items are *the exact same height*.⁷⁶ This is the *fastest* and most efficient option.
2. **prototypeItem:** (e.g., prototypeItem: const MyListItem(...)). This is used when items have dynamic content but are *roughly* the same height.⁶² Flutter will build this *one* prototype widget *in isolation*, measure its dimensions, and then *use that measured size* as the assumed extent for *all* items in the list.

Specifying either "is more efficient... because the scrolling machinery can... know the size of the children *before* they are built".⁶² This turns a complex, per-child, dynamic layout calculation into a simple, static multiplication ($\text{\$offset} = \text{index} \times \text{itemExtent}$), which is dramatically faster and key to achieving smooth scrolling on long lists.

Solution 3: Effective State Management

This is the solution to the `setState()` bottleneck identified in Module 4. We know that calling `setState()` high in the tree is disastrous for performance.⁵⁰ The goal of a proper state management solution (like Provider, Riverpod, BLoC, etc.) is to enable **"selective listening"**.⁷⁹ Instead of a single `StatefulWidget` for an entire screen, state is "lifted" into a separate object (a `ChangeNotifier`, a `Bloc`, etc.). Then, *only* the specific widgets that *depend* on that state are wrapped in a special "listener" widget (like a `Consumer` or `BlocBuilder`).

Consider the "like" button example:

- **The Bad Way (`setState()`):** The "like" button is pressed. It calls `setState()` on the *page*. The *entire page* (`AppBar`, body, 100 `ListItems`) is rebuilt.
- **The Good Way (`Provider/Bloc`):** The "like" button is pressed. It calls a function in the `ChangeNotifier/Bloc`. This object notifies *its listeners*. The *only* listener is the `Consumer` that is wrapped *only* around the `Text` widget displaying the 'like count.'
- **The Result:** The *only* widget that rebuilds is the tiny `Text` widget. The framework has gone from "rebuilding the entire screen"⁵⁸ to "rebuilding one `Text` widget." This is the key to building scalable, high-performance, and efficient UIs.⁸¹

Solution 4: Offloading with Isolates

This is the solution to the "Main Thread Blocking" bottleneck. If an application needs to perform a heavy, synchronous computation—such as parsing a massive JSON file⁸³, processing an image⁸³, or running a complex algorithm⁸⁴—it *cannot* do this on the main UI thread.

The main thread is responsible for the rendering pipeline. If it is "blocked" by a 500ms JSON parsing task, the app *freezes* for 500ms. No frames are rendered, no user input is processed. This is not jank; it is a total application freeze.³⁹

The solution is **Dart Isolates**. An isolate is *not* a "thread" in the traditional Java/C++ sense. While threads typically share memory, **isolates do not share memory**.⁸⁵ Each isolate is an "independent worker" with its own memory heap and its own event loop.⁸⁵ They can only communicate by *passing messages* (similar to platform channels), which ensures there are no data races or locking.

While developers can `spawn()` isolates manually⁸⁵, the framework provides a much simpler helper function for one-off tasks: **`compute()`**.⁸⁴

```
final result = await compute(myHeavyParsingFunction, myJsonString);
```

This function:

1. Spawns a new isolate on-the-fly.
2. Sends the function and its argument to the new isolate.

3. The new isolate performs the heavy work.
 4. The main UI isolate is *unblocked* and free to continue rendering frames 60 times per second.⁸³
 5. When the work is done, the new isolate sends the result back and is shut down.
- This is the standard pattern for offloading any CPU-intensive work to keep the UI "buttery smooth".⁸³

Module 6: Optimizing Flutter for Energy Efficiency

Flutter's Energy Profile

Revisiting the primary energy drivers (Screen, CPU/GPU, Network) through the lens of Flutter reveals specific, framework-related traps. While Flutter's AOT-compiled native code and GPU-accelerated rendering make it highly efficient by default⁵⁰, its "own-the-pixels" architecture creates new developer responsibilities.

1. **CPU/GPU Drain:** Because Flutter *owns* the rendering, an "idle" app with a "decorative animation" (like a pulsating button)²¹ is *not* idle. The developer has *forced* the app to run the CPU and GPU 60 times per second, *indefinitely*. This is a massive, self-inflicted energy drain that would not happen with a native, static widget.²
2. **Network Drain:** The RRC "energy tail" problem²⁷ is platform-agnostic. A Flutter app that makes frequent, small network requests (e.g., for analytics) will drain the battery just as fast as a native app.⁸⁸
3. **Background/Listener Drain:** The StatefulWidget lifecycle includes `dispose()`. A developer who creates a listener (e.g., for GPS or a Timer) and *fails* to cancel it in the `dispose()` method will create an *energy leak*. The app will be closed, but the listener will continue to run in the background, consuming hardware resources and draining the battery.⁹⁰

Energy Tip 1: Taming Animations

Problem: A decorative, looping animation *never* allows the application to become idle. It forces a constant 60 or 120 FPS render cycle, which is a significant tax on the CPU, GPU, and battery.²¹ Users will notice this drain and are 2.5x more likely to uninstall the app as a result.²

Solutions:

1. **Be Respectful:** Avoid "idle" animations. A developer who added a "pulsating button" 'because I could'²¹ created a performance bug. If an animation is purely decorative and runs when the app is idle, it should be removed.

2. **Use TickerMode:** For non-essential animations, Flutter provides a TickerMode widget. This allows an animation to be *dynamically* enabled or disabled.² The professional approach is to use this to *listen* for the device's native "battery saver mode" and *automatically disable* all non-essential animations. Native apps do this; Flutter apps should, too.⁹²
3. **Limit Frame Rate:** If an animation *must* run (e.g., a background visualization), it can be possible to run it at a *lower* frame rate. By providing a custom TickerProvider, an animation can be told to "tick" 10 times per second, not 60, drastically reducing its energy cost.²¹

Energy Tip 2: Batching Network Requests

Problem: This is the application-layer solution to the protocol-layer "energy tail" problem.²⁷ Sending many small, frequent requests (e.g., "user performed action X") is one of the worst offenders for battery drain.⁹⁴

Solution: Batching.

Instead of sending a request immediately, the application should:

1. Store the event (e.g., the user action) in a *local queue*. This can be a simple List in memory or, for a more robust solution, a **local SQLite database**.⁹⁴
2. Use a Timer or a background scheduler to "flush" this queue periodically (e.g., every 60 seconds) or when the app is next started.⁹⁴
3. When the task runs, it takes all 50 queued events and sends them to the server in a *single batch* ⁹⁴, or in parallel using Future.wait.⁹⁵

The energy savings are not linear; they are exponential. Instead of paying the 10-second "energy tail" 50 times (500 seconds of high-power radio), the app pays it *once* (10 seconds). This is a critical optimization for any connected app.

Energy Tip 3: Managing Listeners and Background Processes

Problem: A "zombie" app. The user has "closed" the app, but it is still draining the battery.⁹⁰ This is an *energy leak* caused by a listener that was never properly disposed of. Common culprits are "rogue" GPS listeners ⁹¹ or Timers that are still firing.⁹⁰

Solution 1: The dispose() Method

The StatefulWidget lifecycle provides a dispose() method. It is mandatory for any StatefulWidget that initializes a persistent listener (like a StreamSubscription, a Timer, or a platform channel listener) to cancel that listener in its dispose() method.⁹⁰ This "cleans up" the widget and allows the hardware (like the GPS) to power down.

Solution 2: Intelligent Background Tasks

If an application needs to run code in the background (e.g., to sync data), it should not try to

keep its own process alive. Modern mobile OSes will terminate it. The correct approach is to use a dedicated plugin like workmanager.⁹⁷

This plugin registers the task *with the native* OS (e.g., Android's WorkManager). This *delegates* control to the OS, which can then *intelligently schedule* the task to run at the most energy-efficient time—for example, by batching it with other apps' background work or running it only when the device is charging and on Wi-Fi.⁹⁸ This is for *intermittent* tasks. For *continuous* tasks (like a music player), a foreground service (e.g., via flutter_background_service) is required.⁹⁸

Module 7: The Analyst's Toolkit: Flutter DevTools

Introduction to Flutter DevTools

Performance optimization is an evidence-based science. It is not possible to "guess" where a bottleneck is. The most critical skill for a developer is not to *memorize* optimizations, but to *know how to use* the tools that *find* the bottlenecks.

Flutter provides a first-class, web-based suite of performance and debugging tools called **Flutter DevTools**.⁹⁹

Before any analysis, one step is **CRITICAL: Run in 'Profile Mode'** (flutter run --profile).⁴⁸

- **Debug Mode** (flutter run): **Do not use for profiling.** Performance is *not* indicative of the release app.⁴² It is JIT-compiled and contains extra debugging code.
- **Release Mode** (flutter build): This is the final, AOT-compiled app, but you *cannot* connect DevTools to it.
- **Profile Mode** (flutter run --profile): This is the "Goldilocks" mode. It is AOT-compiled *like a release app* but retains just enough debugging information to allow DevTools to connect.⁴⁸ **Always analyze performance in Profile Mode.**

DevTools provides a suite of tabs for inspecting layout, CPU, memory, and network requests.⁹⁹

DevTools: Performance View and Frame Analysis

The first and most important tab is the **Performance** view.⁴⁸ This view displays the **Flutter Frames Chart**, which is the primary tool for diagnosing jank.

Each bar in the chart represents a single frame rendered by the application. The horizontal line represents the 16ms frame budget.⁴⁸ A bar that extends *above* this line is a **janky frame**, and DevTools will color it *red*.⁶³

Crucially, each frame bar is split into two colors, representing the *two main threads* involved in rendering⁴⁸:

- **Blue (UI Thread):** This represents work done by *your Dart code* in the Dart VM.⁶³ This is the time spent in your `build()`, `layout()`, and `paint()` methods.
- **Green (Raster Thread):** This represents work done by the *Flutter Engine* (Skia/Impeller).⁶³ This is the C++ code that takes the paint commands and communicates with the GPU.

This UI/Raster split is the *primary diagnostic tool*. When a janky red bar appears:

- If the **Blue (UI)** portion is large, the bottleneck is *your Dart code*. This is likely an inefficient `build()` method (due to a bad `setState()` call) or a complex *layout* (due to a deep/large widget tree).
- If the **Green (Raster)** portion is large, the bottleneck is in the *engine*. This is *caused* by your code, but *indirectly*. It is the classic symptom of an *expensive widget* like `Opacity` or `ClipRRect` that is forcing a `saveLayer()` operation.⁶³

This split immediately cuts the diagnostic problem space in half.

DevTools: Reading a Flame Chart (CPU Profiler)

If the **Blue (UI) bar** is red, the next step is to click that frame and open the **CPU Profiler** tab to see *why* the Dart code was slow. This will display a **Flame Chart**.¹⁰³

A flame chart is a visualization of the call stack over time¹⁰⁵:

- **Vertical (Y-axis):** Represents stack depth (e.g., `main()` calls `build()`, which calls `MyWidget.build()`).
- **Horizontal (X-axis):** Represents time. A *wide* bar is a *slow function*.¹⁰⁵

While reading the top-down flame chart is possible, the most efficient analysis uses the **"Bottom Up"** view.¹⁰³ This view inverts the call stack and aggregates the *total time* spent in each function.

The "Bottom Up" tab provides the answer:

1. Go to the "Bottom Up" tab.
2. Sort by "Self Time" (the time spent *in* that function, not its children).
3. The function at the top of this list is the exact function that is consuming the most CPU time.¹⁰³

In a janky Flutter app, this is almost always a specific widget's `build()` method. This tool pinpoints the exact code that needs to be optimized.

DevTools: Memory Profiler

The **Memory** view is used to find *memory leaks* and *memory bloat*.⁹⁹ It displays a real-time graph of the Dart VM's heap. While small, frequent spikes are normal (this is just the GC doing its job⁹⁹), a "sawtooth" pattern of memory that *continuously trends upwards* indicates a memory leak.

To find the source of a leak, the "Diff Snapshots" feature provides a clear, step-by-step

algorithm¹⁰⁶:

1. Navigate to the screen in the app that is suspected of leaking.
2. In DevTools, take a **Heap Snapshot**.
3. Leave that screen (e.g., press the 'Back' button).
4. In DevTools, press the **"Trigger GC"** button to force garbage collection.
5. Take a *second* **Heap Snapshot**.
6. Use the **"Diff Snapshots"** feature to compare Snapshot 1 and Snapshot 2.

In a healthy app, the 'Diff' should be zero or negative; all objects created by that screen should have been garbage collected. If the Diff *still* shows objects from that screen (e.g., MyScreenState, MyController), then a **memory leak** has been confirmed.¹⁰⁷ Some other object (like a rogue listener) is still holding a reference to that screen, preventing it from being 'disposed' and causing the leak.

DevTools: Network Profiler

The **Network** view records every HTTP request made by the application in real-time.¹⁰⁸ This tool is invaluable for *verifying* that both performance and energy optimizations are working correctly.¹¹⁰

Analysts should look for:

1. **Energy (The 'Energy Tail' Problem):** Look at the *frequency* of requests. Is the app sending many small requests every few seconds?¹¹⁰ This is visual *proof* that the app is *not* batching and is *draining the battery*.²⁷ A properly batched app would show a *single* request (or a burst) every 60 seconds.
2. **Performance (Data Size & Caching):**
 - **Size:** Check the *size* of responses. Is the app downloading a 3MB, 3000x3000 pixel image for a 300x300 thumbnail? This is a massive waste of network time, decode time, and memory.¹¹¹
 - **Caching:** Go to a screen, leave it, and then return. Did the *exact same* network requests fire again?⁸⁸ If so, the application's caching strategy is broken, forcing the user to wait for data that should have been available locally.⁸⁹

This tool provides the objective *verification* needed to confirm that the solutions for batching⁹⁴ and caching⁸⁸ have been implemented correctly.

Summary and Key Takeaways

This lecture has explored performance as a holistic system, balancing speed with efficiency.

1. **Performance is a System:** It is a balance of *Speed* (Startup³, Frame Time¹³) and *Efficiency* (Memory¹, Energy¹⁵). These metrics are all interconnected.
2. **Energy is Driven by Hardware:** Application code is in direct control of the device's

biggest battery drains: the **Screen** ¹⁶, the **CPU/GPU** ¹⁶, and the **Network Radio**.²⁷

3. **Flutter's Architecture is a Trade-off:** By "owning the pixels" ³², Flutter gains AOT-compiled native speed ³⁵ but creates new, framework-specific bottlenecks like `saveLayer` ⁵³ (from `Opacity`) and the `setState` rebuild cascade.⁵⁰
4. **Optimize Intelligently (Speed):**
 - Fix jank by *localizing state* (Provider/Bloc) ⁷⁹ and *helping the framework* by using `const` ⁶⁷ and `itemExtent`.⁶²
 - Fix freezes by *offloading heavy work* from the main thread using `compute/Isolates`.⁸⁶
5. **Optimize Respectfully (Energy):**
 - Fix battery drain by *batching* network calls ⁹⁴, *disabling* idle animations ²¹, and *always* `dispose()`ing listeners.⁹⁰
6. **Profile, Profile, Profile:** Optimization is not guesswork. Use **Profile Mode** ⁴⁸ and let **DevTools** ⁹⁹ be the guide.
 - **Red bar in Frame Chart?** -> Jank.
 - **Blue bar?** -> UI Thread (A Dart `build()/layout()` problem).
 - **Green bar?** -> Raster Thread (A GPU `saveLayer()/Opacity` problem).
 - **Wide bar in Flame Chart?** -> The specific slow function.

Questions

- **Further Reading:**
 - Flutter Performance Documentation ⁴²
 - Flutter DevTools Documentation ⁴⁸
 - Android Performance Documentation ¹

Works cited

1. Manage your app's memory | App quality - Android Developers, accessed November 15, 2025, <https://developer.android.com/topic/performance/memory>
2. Make Users Smile: Optimize Your Flutter App's Performance - Dev Siro's Blog, accessed November 15, 2025, <https://dev.siro.co.ke/optimize-your-flutter-apps-performance>
3. Overview of measuring app performance | App quality - Android Developers, accessed November 15, 2025, <https://developer.android.com/topic/performance/measuring-performance>
4. Important Performance Metrics. Before you begin to work with... | by Ben Weiss | Android Developers | Medium, accessed November 15, 2025, <https://medium.com/androiddevelopers/important-performance-metrics-c7dacf018eb3>
5. Overview of app launch times | New Relic Documentation, accessed November

15, 2025,

<https://docs.newrelic.com/docs/mobile-monitoring/new-relic-mobile/get-started/introduction-app-launch-times/>

6. App startup time | App quality - Android Developers, accessed November 15, 2025, <https://developer.android.com/topic/performance/vitals/launch-time>
7. Cold vs. Hot vs. Warm App Launch Times (with Tips for Better UX) | Luciq, accessed November 15, 2025, <https://www.luciq.ai/blog/understanding-cold-hot-and-warm-app-launch-time>
8. Cold, Hot, and Warm App Launch Times and their impact on user engagement | by Umair Khalid | Medium, accessed November 15, 2025, <https://medium.com/@umairkhalid786/cold-hot-and-warm-app-launch-times-and-their-impact-on-user-engagement-d7b759bcc815>
9. Metrics - Apptim Docs, accessed November 15, 2025, <https://docs.apptim.com/guides/metrics>
10. Advanced Game Animation Metrics: Janks And Frametimes - GameBench, accessed November 15, 2025, <https://blog.gamebench.net/advanced-game-animation-metrics-janks-and-frametimes>
11. Mobile Performance Metrics You Should Know — Part 2 | by Sofia Palamarchuk | Medium, accessed November 15, 2025, <https://medium.com/@sofipalamarchuk/mobile-performance-metrics-you-should-know-part-2-8fdc3e5a46dd>
12. Android App Performance: Complete Guide to 120 FPS Optimization - Quash, accessed November 15, 2025, <https://quashbugs.com/blog/android-performance-profiling-from-jank-to-smooth-120fps>
13. Notes for jank and stutter - PerfDog | Full Mobile Platform Performance Test & Analysis Tool, accessed November 15, 2025, https://perfdog.wetest.net/article_detail?id=20&issue_id=0&plat_id=1
14. ARC vs Garbage Collection: The Real Reason Android Needs More RAM Than iOS, accessed November 15, 2025, <https://medium.com/@durgaviswanadh/arc-vs-garbage-collection-the-real-reason-android-needs-more-ram-than-ios-3c654e058aee>
15. The Model Is Not Enough: Understanding Energy Consumption in Mobile Devices - Microsoft, accessed November 15, 2025, <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/phone-hot-chips-2012.pdf>
16. How much does CPU usage affect battery usage? : r/computing - Reddit, accessed November 15, 2025, https://www.reddit.com/r/computing/comments/e78i9/how_much_does_cpu_usage_affect_battery_usage/
17. Optimizing mobile display brightness by leveraging human visual perception, accessed November 15, 2025, <https://www.csl.sri.com/users/jha/papers/cases15.pdf>
18. Dark mode may not save your phone's battery life as much as you think, but there

are a few silver linings - Purdue University, accessed November 15, 2025,
<https://www.purdue.edu/newsroom/archive/releases/2021/Q3/dark-mode-may-not-save-your-phones-battery-life-as-much-as-you-think,-but-there-are-a-few-silver-linings.html>

19. How Does Auto-Brightness Work & Impact Battery Life? - Lenovo, accessed November 15, 2025,
<https://www.lenovo.com/us/en/glossary/how-to-turn-off-auto-brightness-on-pc/>
20. Does auto brightness drains more battery? : r/AndroidQuestions - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/AndroidQuestions/comments/6m0pfv/does_auto_brightness_drains_more_battery/
21. On the energy usage of flutter Animations, and their purpose. : r/FlutterDev - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/FlutterDev/comments/xqy3wl/on_the_energy_usage_of_flutter_animations_and/
22. What is your CPU/GPU/Battery power consumption when gaming on the battery? - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/ASUSROG/comments/18t05gj/what_is_your_cpugpubattery_power_consumption_when/
23. Evaluating the Energy Consumption of Mobile Data Transfer—From Technology Development to Consumer Behaviour and Life Cycle Thinking - MDPI, accessed November 15, 2025, <https://www.mdpi.com/2071-1050/10/7/2494>
24. Why does reducing CPU frequency improve battery life? - Super User, accessed November 15, 2025,
<https://superuser.com/questions/841889/why-does-reducing-cpu-frequency-improve-battery-life>
25. How to Solve 100% CPU Usage, Poor Performance, Poor Battery Life Issues - Efani, accessed November 15, 2025,
<https://www.efani.com/blog/solve-100-cpu-usage-poor-performance-battery-life>
26. Reducing Energy Consumption in Mobile Networks - NEC Corporation, accessed November 15, 2025,
<https://www.nec.com/en/global/techrep/journal/g23/n01/230104.html>
27. Energy Consumption in Mobile Phones: A Measurement Study and Implications for Network Applications - UMass Amherst, accessed November 15, 2025,
<http://www.cs.umass.edu/~arun/papers/TailEnder.pdf>
28. Do Location Tracking Apps Drain Your Battery? The Truth & Tips to Save Power - Lystloc, accessed November 15, 2025,
<https://www.lystloc.com/blog/do-location-tracking-apps-drain-mobile-battery/>
29. Why does GPS in smartphones take so much power if they're just receivers for satellite signals? - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/askscience/comments/1jpsaz/why_does_gps_in_smartphones_take_so_much_power_if/
30. Does Having Location Services On Drain Battery? - Timeero, accessed November 15, 2025,

<https://timeero.com/post/do-gps-tracking-apps-drain-mobile-battery-heres-what-you-need-to-know>

31. Everything You Should Know About GPS Tracking Battery Life - LandAirSea, accessed November 15, 2025, <https://landairsea.com/blogs/consumers/everything-you-should-know-about-gps-tracking-battery-life>
32. Flutter & Skia: Driving High-Performance UI Rendering | by Ronak Jain | Medium, accessed November 15, 2025, <https://medium.com/@ronakofficial/flutter-skia-driving-high-performance-ui-rendering-fcbc6d8ac9ec>
33. How Does Skia Contribute as One of the Key Graphics Engines in Flutter Apps - DhiWise, accessed November 15, 2025, <https://www.dhiwise.com/post/how-does-skia-contribute-as-graphics-engines-in-flutter-apps>
34. Understanding Flutter Architecture — The Power Behind Its Performance | by Dev Sanjana, accessed November 15, 2025, <https://medium.com/@devsanflutter/understanding-flutter-architecture-the-power-behind-its-performance-d2f5a0fee525>
35. Flutter architectural overview, accessed November 15, 2025, <https://docs.flutter.dev/resources/architectural-overview>
36. Flutter's Build Process Explained | by Harsh Kumar Khatri - Medium, accessed November 15, 2025, <https://mailharshkhatri.medium.com/flutters-build-process-explained-b33c2c4b47ae>
37. What is Flutter Widget Tree: A Comprehensive Guide - F22 Labs, accessed November 15, 2025, <https://www.f22labs.com/blogs/what-is-flutter-widget-tree-a-comprehensive-guide/>
38. Building user interfaces with Flutter - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/ui>
39. Fixing Performance Bottlenecks in High-Load Flutter Apps | by Expert App Devs - Medium, accessed November 15, 2025, <https://medium.com/@expertappdevs/fix-performance-bottlenecks-in-flutter-apps-7aaaa51f827e>
40. Impeller rendering engine - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/perf/impeller>
41. A comparison between the old Skia engine and the new impeller engine (for Android) : r/FlutterDev - Reddit, accessed November 15, 2025, https://www.reddit.com/r/FlutterDev/comments/13txm8c/a_comparison_between_the_old_skia_engine_and_the/
42. Improving rendering performance - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/perf/rendering-performance>
43. Comparing AOT and JIT Compilers: Understanding the Differences and Making an Informed Choice — Programming concepts - Amin Nezampour, accessed November 15, 2025,

- <https://aminnez.com/programming-concepts/jit-vs-aot-compiler-pros-cons>
44. AOT vs JIT: Understanding the Java Compiling Processes - CodeStringers, accessed November 15, 2025, <https://www.codestringers.com/insights/aot-vs-jit/>
 45. Unexpected performance differences of JIT/AOT ASP.NET; why? : r/dotnet - Reddit, accessed November 15, 2025, https://www.reddit.com/r/dotnet/comments/1mku4p2/unexpected_performance_differences_of_jitaot/
 46. jit - What are the advantages of just-in-time compilation versus ahead-of-time compilation?, accessed November 15, 2025, <https://stackoverflow.com/questions/2106380/what-are-the-advantages-of-just-in-time-compilation-versus-ahead-of-time-compila>
 47. Are there any performance benefits from a JIT Rust compiler - as it is the case with Dart?, accessed November 15, 2025, https://www.reddit.com/r/rust/comments/g3uanu/are_there_any_performance_benefits_from_a_jit/
 48. Use the Performance view - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/tools/devtools/performance>
 49. Diagnosing Jank with Flutter DevTools Timeline - Vibe Studio, accessed November 15, 2025, <https://vibe-studio.ai/insights/diagnosing-jank-with-flutter-devtools-timeline>
 50. Performance best practices - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/perf/best-practices>
 51. Flutter Performance Optimization: Best Practices with Real Examples | by Dhruv Manavadaria | Medium, accessed November 15, 2025, <https://medium.com/@dhruvmanavadaria/flutter-performance-optimization-best-practices-with-real-examples-912a853c158a>
 52. I am thinking about learning Flutter but the Widget Tree structure feels wrong - Reddit, accessed November 15, 2025, https://www.reddit.com/r/FlutterDev/comments/1eal6sb/i_am_thinking_about_learning_flutter_but_the/
 53. Optimizing Flutter App Performance: Avoid Excessive Use of Opacity and Clipping - Logique, accessed November 15, 2025, <https://www.logique.co.id/blog/en/2025/04/23/optimizing-flutter-apps/>
 54. Flutter App Looking Too Janky : r/flutterhelp - Reddit, accessed November 15, 2025, https://www.reddit.com/r/flutterhelp/comments/1jk512x/flutter_app_looking_too_janky/
 55. Boost Your Flutter App's Performance with Lazy Loading Widgets | by Ayush S. Mathur | Sep, 2025, accessed November 15, 2025, https://medium.com/@ayush_mathur/boost-your-flutter-apps-performance-with-lazy-loading-widgets-594d5d6785f4
 56. Flutter vs Native: Latency Performance Comparison - Digia Studio, accessed November 15, 2025, <https://www.digia.tech/post/flutter-vs-native-latency-performance-comparison>
 57. How fast are platform channels? : r/FlutterDev - Reddit, accessed November 15,

- 2025,
https://www.reddit.com/r/FlutterDev/comments/17g1dz9/how_fast_are_platform_channels/
58. Struggling with Flutter's setState() – Should I Finally Switch? : r/FlutterDev - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/FlutterDev/comments/1jdbh7f/struggling_with_flutter_setstate_should_i/
 59. performance - Flutter setState Function - Stack Overflow, accessed November 15, 2025, <https://stackoverflow.com/questions/73706905/flutter-setstate-function>
 60. Layouts in Flutter, accessed November 15, 2025, <https://docs.flutter.dev/ui/layout>
 61. Flutter increase performance of large number of Positioned Widgets on screen, accessed November 15, 2025,
<https://stackoverflow.com/questions/72854243/flutter-increase-performance-of-large-number-of-positioned-widgets-on-screen>
 62. Work with long lists - Flutter documentation, accessed November 15, 2025,
<https://docs.flutter.dev/cookbook/lists/long-lists>
 63. Flutter performance profiling, accessed November 15, 2025,
<https://docs.flutter.dev/perf/ui-performance>
 64. FAQ - Flutter documentation, accessed November 15, 2025,
<https://docs.flutter.dev/resources/faq>
 65. What are the limitations of Flutter compared to native mobile development? : r/FlutterDev, accessed November 15, 2025,
https://www.reddit.com/r/FlutterDev/comments/1elo1hz/what_are_the_limitations_of_flutter_compared_to/
 66. 🚀 Optimizing Flutter Apps: Performance Best Practices for Today, accessed November 15, 2025,
<https://medium.com/@tamilnambicse/optimizing-flutter-apps-performance-best-practices-for-today-8f32f29937db>
 67. Why Flutter Prefers const with Constant Constructors for Efficiency - DhiWise, accessed November 15, 2025,
<https://www.dhiwise.com/post/why-flutter-prefer-const-with-constant-constructors/>
 68. Flutter Performance Tips (#1): const constructors : r/FlutterDev - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/FlutterDev/comments/d12m5l/flutter_performance_tips_1_const_constructors/
 69. Const Constructors in Dart - Medium, accessed November 15, 2025,
<https://medium.com/@maliashu1794/const-constructors-in-dart-97d7b543beb3>
 70. Does using const in the widget tree improve performance? - Stack Overflow, accessed November 15, 2025,
<https://stackoverflow.com/questions/53492705/does-using-const-in-the-widget-tree-improve-performance>
 71. When should you actually use a RepaintBoundary? - Medium, accessed November 15, 2025,
<https://medium.com/@geraldnuraj/when-should-you-actually-use-a-repaintboundary>

[dary-e81a282602bd](#)

72. Optimizing Flutter UI with RepaintBoundary, accessed November 15, 2025, <https://ms3byoussef.medium.com/optimizing-flutter-ui-with-repaintboundary-2402052224c7>
73. RepaintBoundary class - widgets library - Dart API - Flutter, accessed November 15, 2025, <https://api.flutter.dev/flutter/widgets/RepaintBoundary-class.html>
74. Are there any disadvantages of using RepaintBoundary excessively : r/FlutterDev - Reddit, accessed November 15, 2025, https://www.reddit.com/r/FlutterDev/comments/py2nh0/are_there_any_disadvantages_of_using/
75. Common mistakes with ListViews in Flutter | by Roman Ismagilov - Medium, accessed November 15, 2025, <https://medium.com/@pomis172/common-mistakes-with-listviews-in-flutter-f22e7dacfa7>
76. flutter ListView itemextent - Stack Overflow, accessed November 15, 2025, <https://stackoverflow.com/questions/53007347/flutter-listview-itemextent>
77. Optimizing industrial data displays in Flutter: a deep dive into ListView.Builder and custom RenderObjects, accessed November 15, 2025, <https://www.industrialflutter.com/blogs/optimizing-industrial-data-displays-in-flutter-a-deep-dive-into-listviewbuilder-and-custom-renderobjects/>
78. Why You Should Use PrototypeItem Property in Flutter's ListView.builder | by Jhon Romero, accessed November 15, 2025, <https://medium.com/@jhonmunozromero/why-you-should-use-prototypeitem-property-in-flutters-listview-builder-c0ce1c9e91b7>
79. Flutter Provider State Management for Optimal Performance - DhiWise, accessed November 15, 2025, <https://www.dhiwise.com/post/achieving-optimal-performance-with-flutter-provider-state-management>
80. Approaches to state management - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/data-and-backend/state-mgmt/options>
81. How to efficiently manage state in a large-scale Flutter application? [closed] - Stack Overflow, accessed November 15, 2025, <https://stackoverflow.com/questions/78763640/how-to-efficiently-manage-state-in-a-large-scale-flutter-application>
82. Flutter State Management: Choosing the Right Approach | by Eman Yaqoob - Medium, accessed November 15, 2025, <https://medium.com/@emanyaqoob/flutter-state-management-choosing-the-right-approach-64b2ccf59235>
83. What the Hell Are Flutter Isolates? And Why Should You Care? - DEV Community, accessed November 15, 2025, <https://dev.to/pranta/what-the-hell-are-flutter-isolates-and-why-should-you-care-5hj2>
84. Isolates in Flutter: My Take on Supercharging Heavy Apps | by Mohammad Mahdi | Medium, accessed November 15, 2025, <https://mohammadmahd.medium.com/isolates-in-flutter-my-take-on-superchar>

[ging-heavy-apps-e7d6ec2b1f7e](#)

85. Isolates in Flutter: Boost Performance with Background Processing - Mobisoft Infotech, accessed November 15, 2025,
<https://mobisoftinfotech.com/resources/blog/flutter-development/flutter-isolates-background-processing>
86. Concurrency and isolates - Flutter documentation, accessed November 15, 2025,
<https://docs.flutter.dev/perf/isolates>
87. Boosting Flutter App Efficiency: Best Practices for Performance Optimization - Maxim Gorin, accessed November 15, 2025,
<https://maxim-gorin.medium.com/boosting-flutter-app-efficiency-best-practices-for-performance-optimization-71aa312e64ca>
88. How to Optimize Network Requests in Flutter? | by REIT monero - Medium, accessed November 15, 2025,
<https://medium.com/@juricavoda/how-to-optimize-network-requests-in-flutter-ba8c03de5bfb>
89. Enhancing Network Efficiency in Flutter: Best Practices and Tip - Insight, accessed November 15, 2025,
<https://insight.vayuz.com/insight-detail/enhancing-network-efficiency-in-flutter:-best-practices-and-tip/bmV3c18xNzMzMzUwODU1Mjg2>
90. Flutter app continues running in background and causes unwanted battery drain, accessed November 15, 2025,
<https://stackoverflow.com/questions/67738713/flutter-app-continues-running-in-background-and-causes-unwanted-battery-drain>
91. How to avoid the battery drain while implementing the GPS? - Stack Overflow, accessed November 15, 2025,
<https://stackoverflow.com/questions/5964925/how-to-avoid-the-battery-drain-while-implementing-the-gps>
92. Animations should disable or degrade when battery saver mode is enabled (Android) · Issue #11436 - GitHub, accessed November 15, 2025,
<https://github.com/flutter/flutter/issues/11436>
93. Flutter: Limit frame rate to reduce battery drain - Stack Overflow, accessed November 15, 2025,
<https://stackoverflow.com/questions/76547565/flutter-limit-frame-rate-to-reduce-battery-drain>
94. Methods to Batch Requests In Flutter : r/FlutterDev - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/FlutterDev/comments/1997rxd/methods_to_batch_requests_in_flutter/
95. Optimize Network Performance in Flutter : A Step-by-Step Guide - Stackademic, accessed November 15, 2025,
<https://blog.stackademic.com/optimize-network-performance-in-flutter-a-step-by-step-guide-a9ba216ab8c1>
96. Flutter app draining battery very fast : r/flutterhelp - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/flutterhelp/comments/rvzwlp/flutter_app_draining_batte

- [ry_very_fast/](#)
97. Background processes - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/packages-and-plugins/background-processes>
 98. Managing Background Processes in Flutter: A Guide to Using Workmanager and Flutter Background Service - Insight, accessed November 15, 2025, <https://insight.vayuz.com/insight-detail/managing-background-processes-in-flutter:-a-guide-to-using-workmanager-and-flutter-background-service/bmV3c18xNzQyNTQ3NzIzNDI1>
 99. Flutter and Dart DevTools, accessed November 15, 2025, <https://docs.flutter.dev/tools/devtools>
 100. Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/>
 101. Can I use the memory profiler dev tool on a Flutter app in release mode? - Stack Overflow, accessed November 15, 2025, <https://stackoverflow.com/questions/73857348/can-i-use-the-memory-profiler-dev-tool-on-a-flutter-app-in-release-mode>
 102. Dive into DevTools - YouTube, accessed November 15, 2025, https://www.youtube.com/watch?v=_EYk-E29edo
 103. Use the CPU profiler view - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/tools/devtools/cpu-profiler>
 104. Debugging with Flutter DevTools: A Comprehensive Guide | by aakash - Medium, accessed November 15, 2025, <https://medium.com/easy-flutter/debugging-with-flutter-devtools-a-comprehensive-guide-301f6da6b9d6>
 105. How to read flame chart - Perfsee, accessed November 15, 2025, <https://perfsee.com/docs/source/flamechart>
 106. Use the Memory view - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/tools/devtools/memory>
 107. Mastering Dart & Flutter DevTools — Part 7: Memory View - Medium, accessed November 15, 2025, <https://medium.com/@fluttergems/mastering-dart-flutter-devtools-memory-view-part-7-of-8-e7f5aaf07e15>
 108. Use the Network View - DevTools - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/tools/devtools/network>
 109. Debug and profile your app with Flutter DevTools | Demo - YouTube, accessed November 15, 2025, <https://www.youtube.com/watch?v=J7s5bu49k3U>
 110. Mastering Dart & Flutter DevTools — Part 4: Network View - Medium, accessed November 15, 2025, <https://medium.com/@fluttergems/mastering-dart-flutter-devtools-network-view-part-4-of-8-afce2463687c>
 111. 5 Tips to Optimize Your Flutter App - YouTube, accessed November 15, 2025, <https://www.youtube.com/watch?v=qSXfSPV4yDw>
 112. Performance - Flutter documentation, accessed November 15, 2025, <https://docs.flutter.dev/perf>